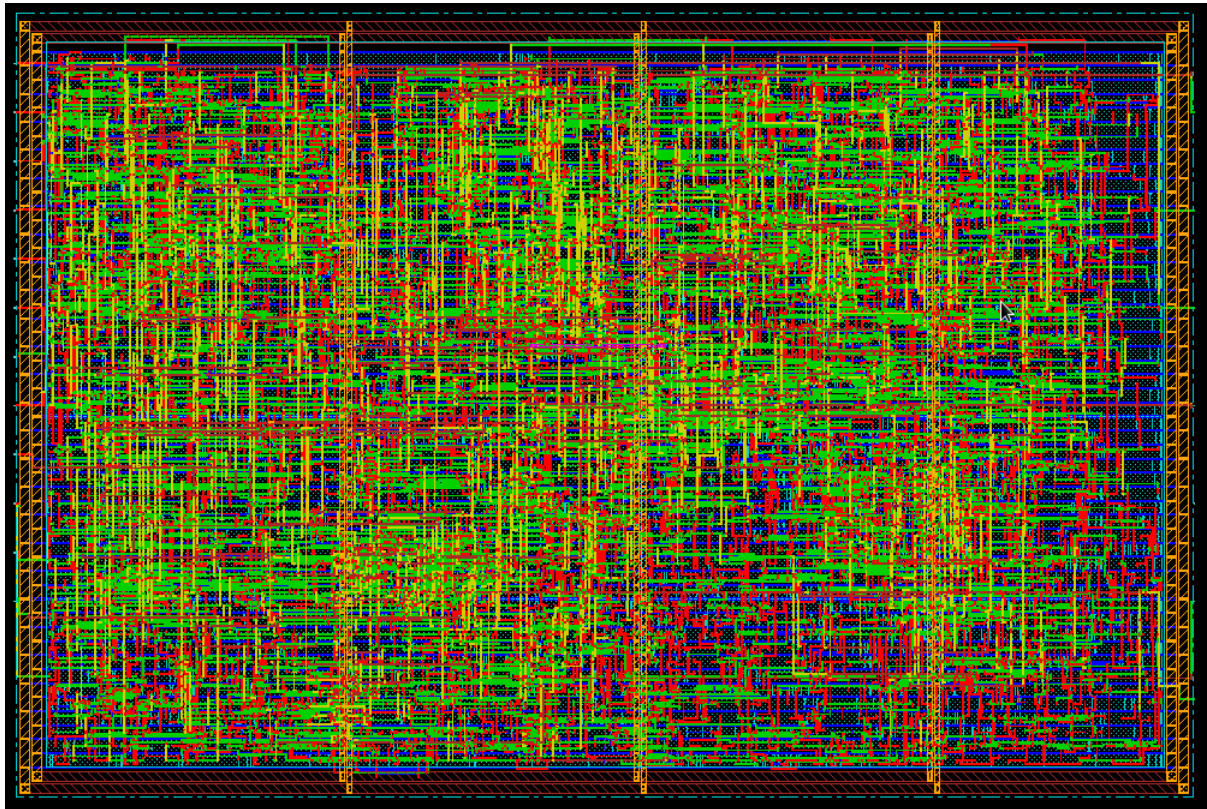


Low-Power Multi-Clock Digital Communication System

From RTL design and verification through synthesis, DFT, STA, formal equivalence and physical implementation



Final routed layout of SYS_TOP (Cadence Encounter)

Author	Hazem Abdelnaby
Flow / Tools	Synopsys Design Compiler, Synopsys Formality, Cadence Encounter, QuestaSim, spyglass

Table of Contents

Executive Summary

1. Introduction

2. System Architecture

3. Design Specifications

4. RTL Design Implementation

5. System Controller Architecture

6. Clock Domain Crossing (CDC)

7. Design for Testability (DFT)

8. Verification Methodology

9. Logic Synthesis

10. Static Timing Analysis (STA)

11. Formal Verification

12. Physical Design (Place and Route)

13. Engineering Challenges and Solutions

14. Results and Performance Evaluation

15. Conclusion

Executive Summary

This report documents the complete RTL-to-GDSII implementation of a low-power, multi-clock digital communication system. The design receives framed commands over a UART interface, decodes and executes them through a central system controller (SYS_CTRL), and returns results to the host through an asynchronous FIFO and a UART transmitter.

Project objectives

- Implement a command-driven UART system capable of register-file access and ALU operations.
- Operate safely across two asynchronous clock domains (REF_CLK at 50 MHz and UART_CLK at 3.6864 MHz) with derived UART clocks.
- Reduce dynamic power through clock gating of the ALU.
- Deliver a testable, timing-clean, physically verified layout suitable for tape-out-style sign-off.

Key design features

- Seven functional blocks integrated in SYS_TOP: UART (RX/TX), SYS_CTRL, register file, ALU, asynchronous FIFO, clock divider/gating and synchronizers.
- Dedicated data and reset synchronizers; Gray-coded pointer exchange in the asynchronous FIFO.
- Runtime-configurable UART baud rate through a prescale-driven clock divider.

Sign-off summary

Metric	Result
Functional verification	pass
DFT (4 scan chains)	99.24%
DRC / LVS	0 / clean

1. Introduction

1.1 Motivation

Serial command interfaces are ubiquitous in embedded SoCs for configuration, debug and data exchange. A realistic implementation must handle clocks that are unrelated in frequency and phase, minimize switching activity, and remain testable after fabrication. This project treats those concerns as first-class requirements and carries the design through the entire ASIC flow rather than stopping at RTL simulation.

1.2 Design objectives and requirements

- Receive UART frames, decode multi-byte commands and dispatch them to the register file or ALU.
- Return read data and ALU results through an asynchronous FIFO to the UART transmitter.
- Support independent system and UART clock domains with robust synchronisation.
- Gate the ALU clock when no operation is pending to reduce dynamic power.
- Insert scan chains and achieve timing closure, formal equivalence and clean physical verification.

1.3 Importance of low-power and multi-clock operation

Multi-clock architectures allow each subsystem to run at the minimum frequency required for its function: the control and compute logic follows REF_CLK while serial I/O runs from UART_CLK-derived clocks. Clock gating removes clock toggling from idle logic, which is typically the dominant contributor to dynamic power in register-rich designs. These choices, however, introduce CDC hazards and complicate DFT and STA, which are addressed in Sections 6, 7 and 11.

1.4 ASIC implementation considerations

Implementation considerations include synchroniser placement, reset-tree structure, scan-compatible clock gating, generated-clock constraints, clock-tree balancing across domains, and equivalence checking in the presence of scan logic and clock-gating cells.

2. System Architecture

2.1 Block diagram

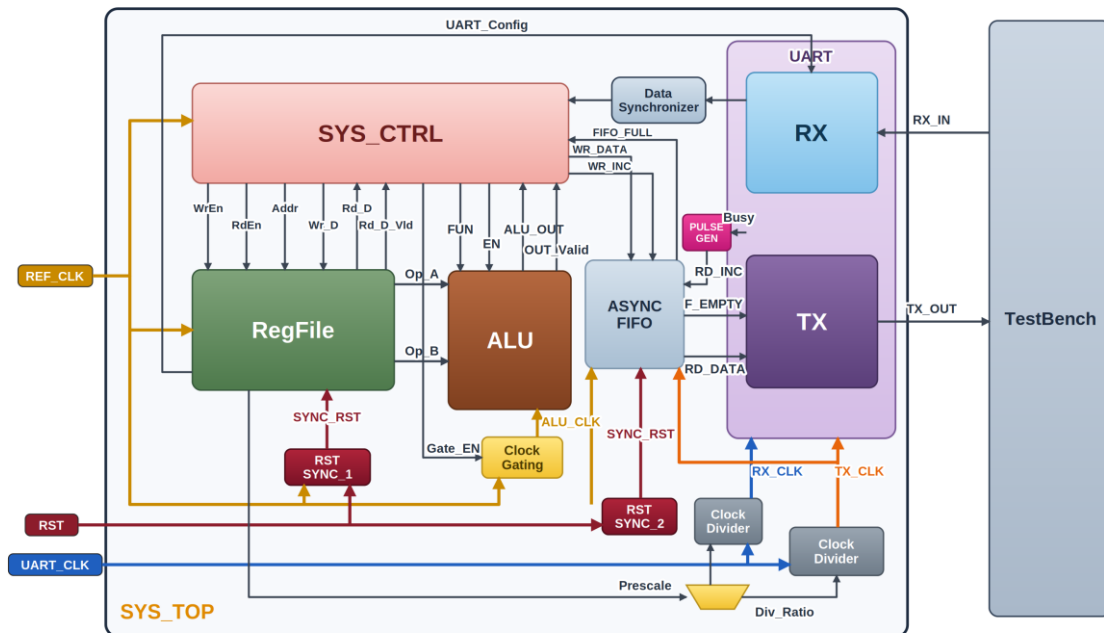


Figure 2.1 – SYS_TOP architecture: clock, reset and data/control paths

2.2 Functional architecture and data flow

UART Receiver. The RX block samples RX_IN using RX_CLK and produces parallel data with a valid indication. Its configuration (UART_Config) is supplied from the register file.

Synchronisation. Because RX_CLK and REF_CLK are asynchronous, received data crosses into the REF_CLK domain through the Data Synchronizer, which converts the RX valid indication into a synchronised enable and captures the data bus safely.

System Controller. SYS_CTRL decodes the command stream and drives the register file (WrEn, RdEn, Addr, Wr_D), the ALU (FUN, EN) and the ALU clock gate (Gate_EN). It consumes Rd_D/Rd_D_Vld and ALU_OUT/OUT_Valid and pushes responses into the FIFO (WR_DATA, WR_INC) while observing FIFO_FULL.

Register File and ALU. The register file stores operands and configuration, supplying Op_A and Op_B to the ALU and the UART configuration and prescale values to the UART and clock-divider logic. The ALU executes the selected function under a gated clock (ALU_CLK).

Asynchronous FIFO. The FIFO decouples the REF_CLK write side from the TX_CLK read side. A pulse generator converts the transmitter Busy status into a single-cycle RD_INC, so one word is popped per transmitted frame.

UART Transmitter. TX reads RD_DATA when F_EMPTY is de-asserted and serialises it on TX_OUT using TX_CLK.

2.3 Clock architecture

Clock	Source	Frequency	Consumers
REF_CLK	External	50 MHz	SYS_CTRL, RegFile, FIFO (write), RST_SYNC_1, Clock Gating
ALU_CLK	REF_CLK via Clock Gating	50 MHz (gated)	ALU
UART_CLK	External	3.6864 MHz	Both clock dividers
RX_CLK	Clock Divider (Prescale)	UART_CLK / prescale	UART RX
TX_CLK	Clock Divider (Div_Ratio)	UART_CLK / Div_Ratio	UART TX, FIFO (read)

2.4 Reset architecture

A single external RST is distributed through two reset synchronisers. RST_SYNC_1 generates SYNC_RST for the REF_CLK domain (register file and system logic); RST_SYNC_2 generates SYNC_RST for the UART/FIFO domain. Each follows asynchronous-assertion, synchronous-deassertion behaviour so that reset release is aligned to the local clock and recovery/removal timing is met.

2.5 Test architecture overview

The design is scan-ready with four scan chains controlled by a test-mode input, a scan-enable signal and dedicated scan clock and reset overrides, so that all clock domains and the gated clock are controllable during shift and capture. Details are given in Section 7.

3. Design Specifications

Parameter	Specification
Data width	8 bits
Register-file address width	4 bits (16 registers)
FIFO depth	8
REF_CLK	50 MHz
UART_CLK	3.6864 MHz
Derived clocks	RX_CLK (prescale), TX_CLK (Div_Ratio), ALU_CLK (gated REF_CLK)
Synchroniser stages	2-flop
Reset methodology	Active-low, asynchronous assert / synchronous de-assert via two RST_SYNC instances
Scan chains	4
Supported commands	Register write, register read, ALU operation with operands, ALU operation without operands

4. RTL Design Implementation

Each module is summarized by objective, function, interface, internal architecture and design considerations.

4.1 UART_RX

Item	Description
Objective	Recover parallel data from the serial RX_IN stream.
Function	Detects the start bit, samples each data bit at mid-bit, optionally checks parity and stop bit, and asserts a data-valid pulse.
Interface	Inputs: CLK(RX_CLK), RST, RX_IN, config (prescale, parity enable/type). Outputs: P_DATA, data_valid, parity_error, stop_error.
Architecture	FSM (idle, start, data, parity, stop) with an edge counter and bit counter; deserialiser shift register; parity and stop checkers.
Design considerations	Mid-bit oversampling for noise margin; error flags kept for the controller; parameterised width.

4.2 UART_TX

Item	Description
Objective	Serialise parallel FIFO words onto TX_OUT.
Function	Frames data with start, optional parity and stop bits, and reports Busy while transmitting.
Interface	Inputs: CLK(TX_CLK), RST, P_DATA, Data_Valid, config. Outputs: TX_OUT, Busy.
Architecture	FSM-controlled serialiser with output multiplexer (start / data / parity / stop) and parity calculator.
Design considerations	Registered outputs to avoid glitches; Busy feeds the pulse generator for FIFO read control.

4.3 UART_TOP

Item	Description
Objective	Integrate TX and RX with shared configuration.
Function	Instantiates UART_RX and UART_TX and exposes a unified configuration interface.
Interface	Ports as per Figure 2.1 (RX_IN, TX_OUT, RX_CLK, TX_CLK, UART_Config, data and handshake signals).
Architecture	Structural wrapper without additional logic.
Design considerations	Keeps clocking of RX and TX independent.

4.4 SYS_CTRL

Item	Description
Objective	Decode UART commands and orchestrate register and ALU transactions.

Item	Description
Function	Frame-based command decoder and sequencer; see Section 5.
Interface	Inputs: synchronised RX data/valid, Rd_D, Rd_D_Vld, ALU_OUT, OUT_Valid, FIFO_FULL. Outputs: WrEn, RdEn, Addr, Wr_D, FUN, EN, Gate_EN, WR_DATA, WR_INC.
Architecture	Single FSM with registered control outputs and an address/command holding register.
Design considerations	Defined behaviour on invalid commands; FIFO back-pressure respected.

4.5 ALU

Item	Description
Objective	Execute arithmetic, logic, comparison and shift operations.
Function	Combinational function units selected by FUN, results registered and flagged by OUT_Valid.
Interface	Inputs: ALU_CLK, RST, EN, FUN, Op_A, Op_B. Outputs: ALU_OUT, OUT_Valid.
Architecture	Hierarchical units (arithmetic, logic, compare, shift) with a decoder and output register.
Design considerations	Runs on a gated clock; wide output for carry/overflow-safe results

4.6 Register File

Item	Description
Objective	Store operands, configuration and prescale values.
Function	Synchronous write, registered read with read-valid indication; dedicated registers drive Op_A, Op_B, UART_Config and Prescale.
Interface	Inputs: CLK, RST, WrEn, RdEn, Addr, Wr_D. Outputs: Rd_D, Rd_D_Vld, Op_A, Op_B, UART_Config, Prescale.
Architecture	Register array with decoded write enable and read multiplexer.
Design considerations	Fixed registers exposed directly to avoid an extra read transaction.

4.7 Clock Divider

Item	Description
Objective	Generate RX_CLK and TX_CLK from UART_CLK.
Function	Programmable integer divider with odd/even ratio support and 50% duty cycle.
Interface	Inputs: I_ref_clk, I_rst, I_clk_en, I_div_ratio. Output: O_div_clk.
Architecture	Counter with toggle flop and odd-ratio compensation; bypass when ratio is 1.
Design considerations	Ratio selected from prescale; generated clocks declared in constraints.

4.8 Clock Gating

Item	Description
Objective	Remove ALU clock activity when idle.

Item	Description
Function	Latch-based integrated clock gate controlled by Gate_EN.
Interface	Inputs: CLK_EN, CLK. Output: GATED_CLK.
Architecture	Level-sensitive latch plus AND gate (or library ICG cell).
Design considerations	Glitch-free enable capture; scan-test enable override.

4.9 DATA_SYNC

Item	Description
Objective	Safely transfer a data bus and its valid strobe between domains.
Function	Multi-flop synchronisation of the enable, pulse generation and data capture on the destination clock.
Interface	Inputs: dest CLK, RST, unsynchronised bus and enable. Outputs: synchronised bus, enable pulse.
Architecture	Two-flop enable synchroniser, edge-detect pulse generator, data register with enable-qualified capture.
Design considerations	Bus held stable by source until the enable is seen; parameterised width and stage count.

4.10 RST_SYNC

Item	Description
Objective	Deliver glitch-free resets aligned to a destination clock.
Function	Asynchronous assert, synchronous de-assert reset synchroniser.
Interface	Inputs: CLK, RST. Output: SYNC_RST.
Architecture	Parameterised flop chain with async-reset flops.
Design considerations	Two instances, one per clock domain.

4.11 Pulse Generator

Item	Description
Objective	Create a single-cycle RD_INC from the Busy status.
Function	Rising-edge detector with a registered previous value.
Interface	Inputs: CLK, RST, Busy. Output: RD_INC.
Architecture	One flop and a gate.
Design considerations	Ensures exactly one FIFO pop per frame.

4.12 ASYNC_FIFO

Item	Description
Objective	Transfer response data from REF_CLK to TX_CLK.

Item	Description
Function	Dual-clock FIFO with Gray-coded pointers, two-flop pointer synchronisers and full/empty flag generation.
Interface	Inputs: W_CLK, W_RST, W_INC, WR_DATA, R_CLK, R_RST, R_INC. Outputs: RD_DATA, FIFO_FULL, F_EMPTY.
Architecture	Dual-port memory, write and read pointer logic, binary-to-Gray converters, pointer synchronisers.
Design considerations	Pointers one bit wider than the address for wrap detection; only Gray pointers cross domains.

5. System Controller Architecture

5.1 FSM overview

SYS_CTRL is a command-driven FSM operating in the REF_CLK domain. It remains in IDLE until the Data Synchronizer reports a valid received byte, decodes the first byte of the frame as a command, and then advances through a command-specific sequence of states, consuming one received byte per transition. All outputs are registered to ensure clean timing and glitch-free control of the register file, ALU and FIFO.

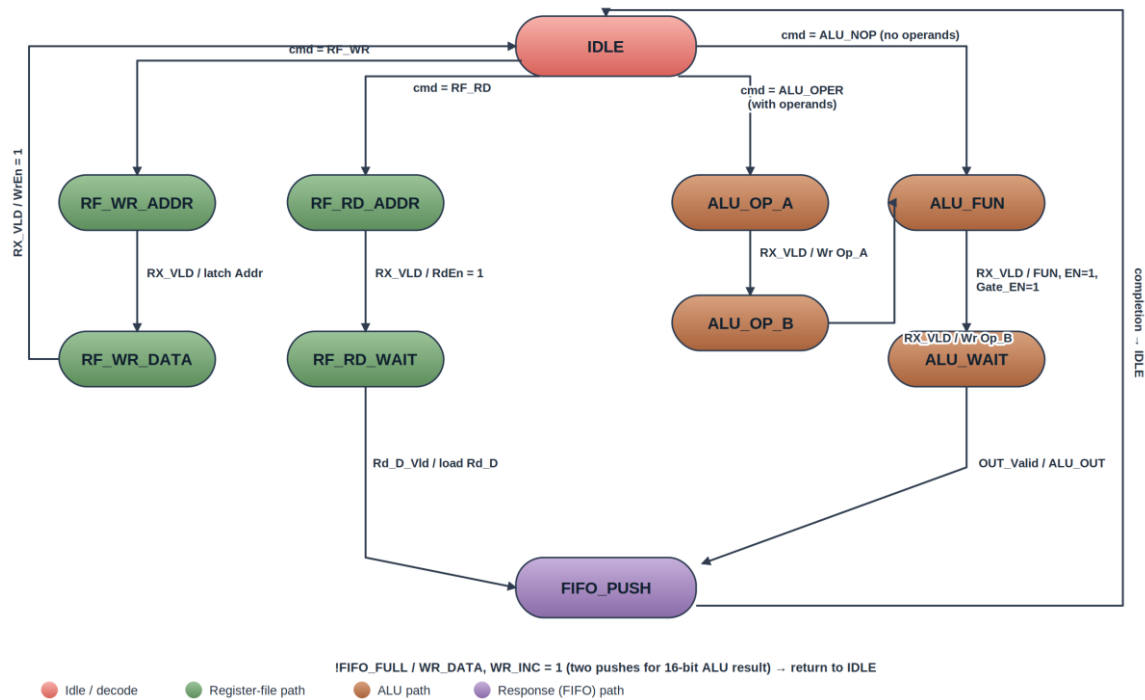


Figure 5.1 – SYS_CTRL state diagram (reference architecture; align state names with the RTL)

5.2 Command decoding and transitions

Command	Frame sequence	Action
Register write	CMD, Address, Data	Latch address, then assert WrEn with Wr_D
Register read	CMD, Address	Assert RdEn, wait for Rd_D_Vld, push Rd_D to FIFO
ALU with operands	CMD, Op_A, Op_B, FUN	Write operands to registers, enable clock gate and ALU, push result
ALU without operands	CMD, FUN	Use stored operands, enable clock gate and ALU, push result

5.3 Operation flows

Register access. For a write, the address byte is held until the data byte arrives, then WrEn is pulsed for one cycle. For a read, RdEn is pulsed and the controller waits for Rd_D_Vld before forwarding Rd_D to the FIFO.

ALU operation. Operands are written to the dedicated operand registers, Gate_EN enables ALU_CLK, and EN together with FUN launches the operation. The controller waits for OUT_Valid, captures ALU_OUT and pushes it into the FIFO; the gate is released afterwards to save power.

UART communication and FIFO control. Responses are written with WR_DATA/WR_INC only when FIFO_FULL is low; otherwise the controller stalls in the push state. A multi-byte ALU result is pushed as consecutive words.

Error handling. Unrecognised commands return the FSM to IDLE without side effects.

5.4 Control signal generation

Signal	Asserted in	Purpose
WrEn / RdEn	Register write / read states	Register-file access strobes
Addr, Wr_D	Register states	Address and data to register file
FUN, EN	ALU function state	Select and launch ALU operation
Gate_EN	ALU path, until OUT_Valid	Enable ALU_CLK
WR_DATA, WR_INC	FIFO push state	Write response word into FIFO

6. Clock Domain Crossing (CDC)

6.1 Clock domains and challenges

The design contains a REF_CLK domain (SYS_CTRL, RegFile, ALU, FIFO write side), an RX_CLK domain (UART RX), a TX_CLK domain (UART TX, FIFO read side) and the UART_CLK source feeding the dividers. Signals that cross between these domains risk metastability, data incoherency and missed pulses if not properly synchronised.

6.2 Crossing structures

- **DATA_SYNC** – RX data/valid to REF_CLK: multi-flop enable synchroniser, pulse generation and enable-qualified data capture, with the bus held stable by the source.
- **RST_SYNC** – asynchronous assertion, synchronous de-assertion in each destination domain.
- **ASYNC_FIFO** – Gray-coded read/write pointers synchronised through two-flop chains; only one bit changes per increment, so any sampled value is either old or new, never invalid.
- **Pulse generator / Busy** – status from TX_CLK domain converted to a local pulse for the read pointer.

6.3 CDC verification methodology

Static CDC analysis was run on the RTL with the clock and reset definitions of Section 3 to classify every crossing (synchronized, unsynchronized, reconvergent, multi-bit) and verify synchronizer structure. Dynamic checks in simulation complemented this with randomized clock phase relationships

7. Design for Testability (DFT)

7.1 Objectives and methodology

The DFT goals are full controllability and observability of sequential elements, high stuck-at fault coverage and compatibility with multiple clock domains. Scan insertion replaces functional flops by scan flops with a multiplexed scan input, stitched into chains and controlled by scan enable.

7.2 Scan architecture

Test signal	Function
test_mode	Places the design in test configuration; overrides functional clock and reset sources
scan_clk	Single shift/capture clock replacing divided and gated clocks in test mode
scan_rst	Test-mode reset so that flops are controllable and not driven by the synchronisers
SE (scan enable)	Selects shift (1) or capture (0) at each scan flop
SI[3:0] / SO[3:0]	Four scan-chain inputs and outputs

Four chains are used to reduce shift time. Chains are balanced by flop count across domains to keep scan-shift length uniform; 100. In test mode, the clock dividers and clock gate are bypassed or forced so that every flop is clocked by scan_clk, and the reset synchronisers are bypassed to remove uncontrolled resets.

7.3 DFT report analysis

Initial violation	Root cause	Debug approach	Fix
Uncontrolled reset (D-type rule)	Reset driven by synchroniser flops, not by a primary input in test mode	Traced reset net to RST_SYNC outputs	Added test-mode multiplexer selecting scan_rst
Uncontrolled / gated clocks	Divided and gated clocks not driven from a test clock	Traced clock nets in DRC report	Added test-mode clock multiplexers; clock-gate test enable

8. Verification Methodology

Verification combines module-level and system-level simulation in QuestaSim/Icarus Verilog. Testbenches are self-checking: expected values are computed or read from reference files and compared automatically, with a pass/fail summary at the end of each run. Directed tests cover each command, and corner cases cover boundary conditions (full/empty FIFO, back-to-back commands, invalid commands, all ALU functions, address extremes, parity and stop-bit errors).

Testbench	Objective	Stimulus	Expected result
UART_TX_TB	Verify framing, parity and Busy timing.	Parallel data patterns with parity enabled/disabled and both parity types.	Serial frame matches expected bit sequence, polarity and timing.
UART_RX_TB	Verify sampling, deserialization and error detection.	Serial frames driven at the configured baud rate including corrupted parity/stop.	Correct parallel data, valid pulse and error flags.
DATA_SYNC_TB	Verify safe crossing and pulse generation.	Random enable timing against destination clock.	Single pulse per enable; data captured intact.
ASYNC_FIFO_TB	Verify ordering, full/empty and wrap-around.	Independent write and read clocks with randomised rates.	No data loss or duplication; flags assert correctly.
ALU_TB / RegFile_TB	Verify every function and register access.	Directed and random operand/address sets.	Outputs equal reference model.
System_TOP_TB	Verify end-to-end command execution.	UART frames for register write/read and ALU operations with and without operands.	Response frames on TX_OUT equal expected values.

9. Logic Synthesis

Synthesis was performed with Synopsys Design Compiler. The flow reads the RTL, applies the SDC constraints, compiles with the target library, inserts scan (Section 7), and writes the netlist, SDC and reports used by STA, Formality and PnR.

Item	Detail
Libraries	SS for setup analysis FF for hold analysis
Constraints	Clock definitions (REF_CLK 50 MHz, UART_CLK 3.6864 MHz), generated clocks for RX_CLK/TX_CLK/ALU_CLK, clock groups, I/O delays, driving cell and load
Optimisation strategy	Mapping high effort

Constraint development followed an iterative approach: a baseline clock-only SDC was refined with generated-clock definitions and clock groups once the divider and gating structure was finalised, and I/O constraints were derived from the interface timing of the testbench.

10. Static Timing Analysis (STA)

Constraint	Application
Clocks	REF_CLK and UART_CLK defined on their ports with uncertainty and transition
Generated clocks	RX_CLK and TX_CLK from the clock dividers; ALU_CLK through the clock gate
Clock groups	REF_CLK, RX_CLK and TX_CLK declared asynchronous
False paths	Asynchronous reset assertion, static configuration registers, test-mode signals

Setup analysis verifies that data arrives before the capture edge minus setup time; hold analysis verifies stability after the capture edge and is the principal driver of post-CTS buffer insertion. Scan clock interactions are checked in a separate test-mode analysis.

Metric	Result
Optimisation applied	sizing, buffering, restructuring, constraint fixes
Final status	met

11. Formal Verification

Synopsys Formality compares the RTL (reference) against the synthesised netlist (implementation) to prove logical equivalence. The flow reads both designs with their libraries, applies any SVF guidance produced by synthesis, sets constants for test-mode signals, matches compare points, and verifies.

Metric	Result
Reference / implementation	RTL / post-synthesis (and post-PnR) netlist
Unmatched points	0
Failing points	0
Result	Succeed

12. Physical Design (Place and Route)

Physical implementation was performed in Cadence Encounter from the synthesised netlist and SDC.

12.1 Floorplanning

Core utilization: 74%. landscape core, see Figure 13.1). No hard macros are present; the design is fully standard-cell. I/O pins are placed  Power planning follows Section 12.2.

12.2 Power Distribution Network

A power ring surrounds the core and vertical power stripes are distributed across the width to limit IR drop. Standard-cell rails are connected to the stripes and ring by special routing (sroute) 6 metal layers

12.3 Placement

Timing-driven placement with congestion awareness was followed by pre-CTS optimization. Congestion

12.4 Clock Tree Synthesis

CTS builds balanced trees for each clock, including the gated ALU_CLK and the divided UART clocks, honoring generated-clock relationships.

12.5 Routing

Global routing was followed by detailed routing with timing-driven and crosstalk-aware options. DRC considerations include spacing, minimum area, via enclosure and antenna rules. Routing optimization

12.6 post-route optimization

Post-route optimization fixed setup violations through cell sizing and buffering, and hold violations through delay-buffer insertion. Signal-integrity checks assessed crosstalk-induced delay and noise. Res

12.7 Physical verification

Metric	Result
DRC	0 violations
LVS	clean
Connectivity / antenna	0 violations

12.8 Final layout results

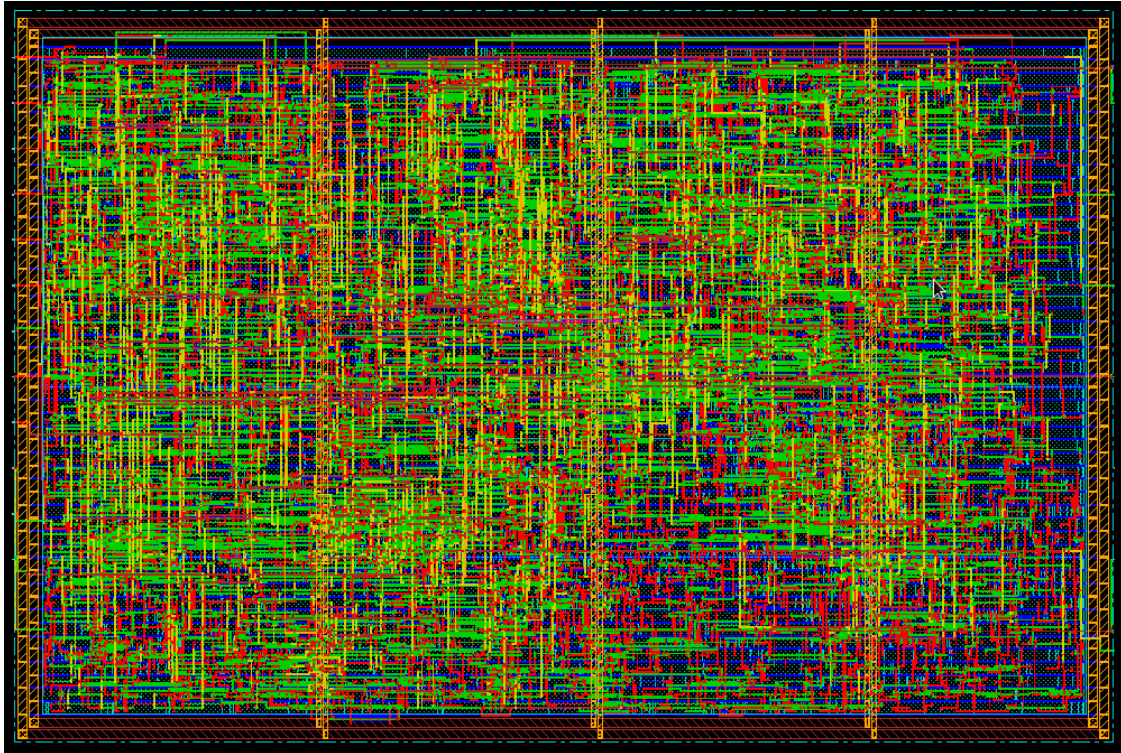


Figure 13.1 – Final routed layout of SYS_TOP showing power ring, stripes and routed metal layers

Metric	Value
utilization	100
Timing status	met

13. Engineering Challenges and Solutions

Each challenge is documented with description, root cause, investigation, solution, outcome and lesson learned. The structure below is pre-filled with the challenges typical of this architecture; edit to match what actually occurred.

13.1 CDC – multi-clock, FIFO and reset synchronisation

Item	Description
Problem	Unsynchronized or multi-bit crossings flagged between RX_CLK, REF_CLK and TX_CLK.
Root cause	Direct use of RX valid/data in REF_CLK logic; binary pointers or resets crossing domains.
Investigation	Reviewed CDC report crossings and traced sources/destinations.
Solution	DATA_SYNC for the bus and strobe, Gray-coded FIFO pointers, per-domain RST_SYNC.
Outcome	CDC clean with justified waivers only
Lesson learned	Every crossing needs an explicit, verifiable structure.

13.2 DFT – uncontrolled resets, insertion and balancing

Item	Description
Problem	DFT DRC reported uncontrolled resets and clocks; unbalanced chain lengths.
Root cause	Reset from synchronisers and divided/gated clocks not controllable in test mode.
Investigation	Inspected DRC messages and traced nets to their sources.
Solution	Test-mode multiplexers for clock and reset; four chains balanced across domains.
Outcome	DFT DRC clean
Lesson learned	Design for test from the start of RTL.

13.3 Timing closure – setup, hold, generated clocks, scan

Item	Description
Problem	Setup/hold violations and incorrect timing on divided clocks.
Root cause	Missing generated-clock definitions; hold exposed after CTS; scan clock paths.
Investigation	Compared reports before and after constraint refinement.
Solution	Correct generated clocks and groups; sizing/buffering; hold-fix buffers post-CTS.
Outcome	Timing met
Lesson learned	Constraints are part of the design.

13.4 Formality – equivalence and constraint mismatches

Item	Description
Problem	Unmatched or failing points after synthesis with scan.
Root cause	Test-mode signals unconstrained; clock-gating and scan logic differing from RTL.
Investigation	Examined unmatched/failing points and SVF.
Solution	Applied constants and SVF guidance.
Outcome	Equivalence achieved
Lesson learned	Keep tool setups consistent.

13.5 Physical design – congestion, CTS, routing, skew

Item	Description
Problem	Routing congestion and skew across domains.
Root cause	Dense logic around controller, multiple clock trees.
Investigation	Reviewed congestion maps and clock-tree reports.
Solution	Adjusted utilisation, CTS settings and routing options.
Outcome	Clean layout
Lesson learned	Early floorplan decisions affect closure.

14. Results and Performance Evaluation

Aspect	Assessment	Evidence
Functional correctness	All commands verified at module and system level	Section 8, 9
CDC robustness	All crossings use proven structures	Section 6
Testability	Four-chain scan, test-mode control of clocks/resets	Section 7
Timing closure	Setup and hold met across analyzed corners	Section 11, 13
Physical quality	Clean DRC/LVS, balanced clock trees	Section 13
Scalability	Parameterized widths and depths; configurable baud rate	Section 4
ASIC readiness	Complete flow from RTL to layout with sign-off checks	All

15. Conclusion

The project delivered a complete low-power, multi-clock UART command system from RTL to routed layout. The architecture combines a command-driven controller, safe clock-domain crossings, clock gating for power reduction and a four-chain scan architecture, and was taken through synthesis, STA, formal equivalence and physical design.

- **Design quality:** modular, parameterized RTL with registered controller outputs.
- **Verification:** self-checking directed and corner-case testbenches at module and system level.
- **DFT and physical readiness:** scan-ready design with timing-clean, physically verified layout.
- **Key lessons:** treat CDC, DFT and constraints as design inputs from day one.